

Git GUI 與 GitHub 雙向同步全面指南

整合實戰衝突、底層邏輯與問答精華之技術對談紀錄

前言

本文件完整收錄並整理了關於如何使用 Git GUI 圖形化介面與 GitHub 雲端進行雙向同步的操作流程。同時，針對實戰中因軟體暫存檔（如 Obsidian 的 `workspace.json`）引發的衝突、Git 的安全保護機制、以及 `Already up to date` 等核心觀念進行了深度解構，旨在建立一個不論面對任何狀況都能冷靜排除的完整知識庫。

第一部分：上傳流程（本地端 → GitHub 雲端）

當您在本地端完成檔案（如筆記、程式碼）的撰寫或修改，欲同步備份至雲端時，請遵循以下標準四步驟：

- 掃描並檢查變更 (Rescan)**：開啟 Git GUI，點擊左下角的 `Rescan` 按鈕（快捷鍵 `F5`）。此時左上角橘色區域 `Unstaged Changes` 會顯示所有被改動但尚未被追蹤的檔案。
- 移至暫存區 (Stage Changed)**：點擊左下角的 `Stage Changed` 按鈕（快捷鍵 `Ctrl + T`）。檔案將移動至左下角綠色區域 `Staged Changes (Will Commit)`，代表已進入準備打包的名單中。
- 撰寫備忘並打包 (Commit)**：在右下角的 `Commit Message` 文字框中輸入本次修改的重點（例：`v2 修正筆記`），隨後點擊 `Commit` 按鈕。此時綠色區域檔案清空，代表已成功存入本地倉庫的歷史紀錄。
- 推上雲端 (Push)**：點擊右下角的 `Push` 按鈕。確認彈出視窗中的 `Source Branches` 為您當前使用的分支（如 `main` 或 `master`），點擊 `Push`，等待進度條完成並顯示 `Success` 即大功告成。

第二部分：下載流程（GitHub 雲端 → 本地端）

原生 Git GUI 為確保安全性，並未提供「一鍵 Pull」的直覺按鈕，而是將其拆解為兩個安全步驟：

1. 標準 GUI 操作法

• 步驟一：擷取雲端狀態 (Fetch)

點選頂部選單的 `Remote` → `Fetch from` → `origin`。Git 會將雲端最新的歷史紀錄與檔案狀態抓取到本機快取中，此時本機資料夾尚未被修改。完成後點擊 `Close`。

• 步驟二：融合至本地資料夾 (Merge)

點選頂部選單的 `Merge` → `Local Merge...`。在 `Tracking Branches` 清單下選取 `origin/main`（依雲端分支而定），最後點擊 `Merge`。雲端檔案此時才正式與本機檔案進行合併與更新。

2. 命令列快速捷徑

若覺得 GUI 連續點選較為繁瑣，可直接在專案目錄開啟 PowerShell 並輸入以下指令，即可同時自動完成 Fetch 與 Merge：

```
git pull origin main
```

第三部分：實戰常見錯誤與排除方案

此處整合了在實際同步過程中，因 Git 安全保護機制所發生的三大痛點及對應解法：

痛點一：下載時因本地變更未處理而遭自動卡住

【情境還原】輸入 `git pull` 時，Git 終止下載並噴出錯誤：

Your local changes to the following files would be overwritten by merge: ...

【原因解析】以 Obsidian 的 `workspace.json`（視窗佈局暫存檔）為例，只要開啟軟體，該檔案就會隨時變動。Git 偵測到雲端有新版本要下載，但本機的版本也有變動且尚未打包，為了防止雲端檔案直接覆蓋並抹煞您本機的當前變更，因此緊急煞車。

【解決方案】若確定本機的暫存變更不重要（如視窗排版），可在 PowerShell 輸入以下指令強制還原該檔案，使其回復到上一次同步的狀態，再行 Pull：

```
git restore AI/.obsidian/workspace.json
```

痛點二：遭遇 Merge Conflict（合併衝突）導致倉庫鎖定

【情境還原】執行 Pull 後，系統提示 `CONFLICT (content): Merge conflict in...`，Git GUI 畫面出現紅藍相間的驚嘆號，且不允許進行其他操作。

【原因解析】雲端與本地端同時對同一個檔案進行了不同的修改，Git 在嘗試自動融合成單一檔案時失敗，不知該採用哪一方的內容，故將倉庫鎖定。

【三步驟暴力解鎖方案】由於 `workspace.json` 僅是軟體介面暫存檔而不含核心筆記，無須逐行人工對齊，可直接下令以本地版本為主並強行結案：

```
git checkout --ours AI/.obsidian/workspace.json
git add AI/.obsidian/workspace.json
git commit -m "Resolve workspace.json conflict"
```

痛點三：PowerShell 畫面出現 `>>` 符號導致集體凍結

【原因解析】在複製貼上指令時，不小心夾帶了未閉合的引號（如單引號或雙引號）或特殊的換行字元，導致 PowerShell 誤判指令尚未輸入完畢，而在原地等待。

【解決方案】於鍵盤按下 `Ctrl + C` 強制終止當前狀態，重新乾淨地輸入正確指令即可。

第四部分：深度觀念釐清與邏輯思維

💡 觀念一：解鎖衝突時的三行指令，本質到底是什麼？

在痛點二中使用的三行解鎖指令，其核心精髓在於「在本地端為上次卡住的合併程序補辦登記結案手續」。它們並沒有任何將檔案上傳（Push）至雲端的動作：

- `git checkout --ours <file>`：並非從雲端覆蓋下來，而是命令 Git 丟棄雲端抓下來的版本，強制保留本地電腦的版本。
- `git add <file>`：將處理好的檔案放入暫存區，蓋上「衝突已解決」的標記。
- `git commit`：並非上傳，而是為了「幫下載後的合併結果簽字畫押」。告訴 Git 這次聯姻的最終版本長這樣，正式將合併結果存檔並解開倉庫鎖定。若要同步至雲端，必須在解鎖後額外追加執行 `git push origin main`。

💡 觀念二：對 `Already up to date` 的精準理解與優化定義

假設 `a.md` 或 `workspace.json` 先前已 Push 至雲端，且雲端隨後無人動過。此時僅本地端有增修檔案，若執行 `git pull`，將會成功通過並顯示 `Already up to date`。

檔案狀態變化： 本地端剛修改好的內容會原封不動、完好無損地保留。雲端舊的檔案不會蓋掉本地新的檔案。

資訊揭露的優化思考：

系統提示	Git 底層邏輯與思維	使用者體驗感受
Git 原廠顯示： <code>Already up to date</code>	從「時空歷史圖（Commit Tree）」的觀點切入。比對後發現本地目前的版本進度已等於（或超越）雲端最新進度，故判定狀態已同步。	語意過於抽象與工程師腦。使用者常困惑：「你剛剛到底有沒有幫我上網抓檔案？」顯得較為潦草。
最直覺的定義： 雲端查無更新，資料檔案未下載	從「實際檔案搬運與傳輸」的觀點切入。直接告知使用者線上沒有任何新的進度需要被下載，因此本地檔案未受任何改動。	清晰明瞭、踏實且令人安心。明確知道執行成功，且本機心血沒有被覆蓋。

第五部分：雙向變動交叉矩陣與黃金鐵律

Git 同步機制的黃金鐵律：`git pull` 只有在「雲端有本地端所沒有的新 Commit 歷史紀錄」時，才會發動實際的下載與融合。

情境	雲端狀態 (GitHub)	本地狀態 (您的電腦)	執行 <code>git pull</code> 的真實結果	本地檔案的最終命運
1. 雲端無新進度	完全沒人動	有修改、有增修 (如 <code>a.md</code>)	綠燈放行，回報 <code>Already up to date</code>	安全 。本地最新修改完好無損，未發生覆蓋。
2. 雙向皆有變動 (尚未 Commit)	有人 Push 新進度	有修改，但尚未點擊 Commit 打包	自動踩煞車，拒絕執行並噴出 <code>error: ... overwritten by merge</code>	被保護 。Git 拒絕執行，直到您還原檔案或完成 Commit。
3. 雙向皆有變動 (已經 Commit)	有人 Push 新進度	有修改，且已在 Git GUI 點擊 Commit	檔案成功下載，但立刻觸發 <code>Merge Conflict</code> (合併衝突)	被鎖定 。兩邊內容撞在一起，等待您手動修剪並 Commit 結案。

第六部分：一勞永逸的終極防禦 — `.gitignore`

諸如 `workspace.json` 這類隨操作介面即時變動的環境暫存檔，本質上並無任何備份或團隊協作同步的價值。不斷進行同步只會徒增雙向衝突的機率。

建議配置步驟：

在您專案的根目錄（例如 Obsidian 儲存庫資料夾的第一層）建立一個名為 `.gitignore` 的純文字檔案（注意前面有一個小數點），並在檔案內寫入以下規則：

```
.obsidian/workspace.json
```

儲存後，Git 從此便會徹底無視該檔案的任何變動。無論您如何頻繁地開啟、關閉軟體或調整介面，它都不會再出現在 Git GUI 的掃描清單中，讓雙向上下傳流程回歸最純粹、順暢的狀態。